

Programa de Evaluación Práctica: Arquitectura Fullstack e-Commerce

Modalidad: Exámenes prácticos individuales

Condición inicial: Código limpio (*scratch*) o rama base del ejercicio anterior

Stack tecnológico: Node.js, Express, Sequelize, MySQL, React, React Router DOM, Axios, Context API, Tailwind CSS

Estructura General de la Serie

[Ejercicio 1: Back Core] ——> Modelos separados (Admin / Client) + JWT + Middlewares
|
▼
[Ejercicio 2: Front Core] —> AuthContext Dual + Redirección + Rutas Protegidas
|
▼
[Ejercicio 3: RBAC Admin] —> Roles jerárquicos (Superadmin, Gestor, Auditor) + AdminGuard
|
▼
[Ejercicio 4: Data Flow] ——> CRUD Productos + Paginación + Orden + Filtros (Back & Front)
|
▼
[Ejercicio 5: Sesiones] ———> Refresh Token + Persistencia F5 + Aislamiento de Entornos
|
▼
[Ejercicio 6: UI Context] —> ThemeContext + Header Reactivo Multi-rol + Persistencia
|
▼
[Ejercicio 7: Layout UI] ——> Home Mobile-First Responsivo con Tailwind CSS

Ejercicio 1: Backend Core — Autenticación Base con Modelos Separados

Tipo: 100% Backend

Herramienta de prueba: Postman o Thunder Client

1. Contexto y Objetivos

Construir una API REST desde cero que gestione la autenticación de dos entidades totalmente independientes dentro de una tienda online: los **Clientes** (compradores) y los **Administradores** (operadores del sistema). No se deben compartir tablas ni rutas de acceso para evitar colisiones de datos.

2. Modelado de Datos (Sequelize + MySQL)

- **Modelo** `Client` **(**`clients`**):**
    - `id`: Entero, autoincremental, clave primaria.
    - `nombre`: String, obligatorio.
    - `email`: String, obligatorio, único.
    - `password`: String, obligatorio (hasheado con `bcryptjs`).
    - `direccion`: String, opcional.
    - `telefono`: String, opcional.
- **Modelo** `Admin` **(**`admins`**):**
    - `id`: Entero, autoincremental, clave primaria.
    - `nombre`: String, obligatorio.
    - `email`: String, obligatorio, único.
    - `password`: String, obligatorio (hasheado con `bcryptjs`).

Semilla rápida sugerida: Crear un bloque de inicialización con `sequelize.sync({ alter: true })` que inserte por defecto:

- Admin: `admin@tienda.com` / `admin123`
- Cliente: `cliente@tienda.com` / `cliente123`

3. Endpoints a Implementar

Método	Ruta	Acceso	Descripción
`POST`	`/api/auth/client/login`	Público	Busca en `clients`. Retorna token con payload `{ id, type: 'client' }`.

POST	/api/auth/admin/login	Público	Busca en admins. Retorna token con payload { id, type: 'admin' }.
GET	/api/client/perfil	Privado	Valida token y exige type === 'client'. Retorna datos del cliente.
GET	/api/admin/metricas	Privado	Valida token y exige type === 'admin'. Retorna datos administrativos.

4. Middlewares Requeridos

1. verifyToken: Extrae el header Authorization: Bearer <token>, verifica la firma del JWT y adjunta la información decodificada en req.user. Si no hay token o expiró, retorna HTTP 401 Unauthorized.
2. isClient: Verifica que req.user.type === 'client'. Si no lo es, retorna HTTP 403 Forbidden.
3. isAdmin: Verifica que req.user.type === 'admin'. Si no lo es, retorna HTTP 403 Forbidden.

Ejercicio 2: Frontend Core — AuthContext Dual y Rutas Protegidas

Tipo: 100% Frontend
Base: Backend del Ejercicio 1 en ejecución

1. Contexto y Objetivos

Construir la aplicación en React conectándola a la API anterior. El sistema debe permitir iniciar sesión como cliente o como administrador, guardar la sesión de forma global mediante Context API y proteger el árbol de rutas evitando cruce de portales.

2. Especificación de Rutas (`react-router-dom`)

- `/`: Vista pública con enlaces para ir a Login Cliente o Login Admin.
- `/login`: Formulario de acceso para clientes.
- `/admin/login`: Formulario de acceso para administradores.
- `/cliente/dashboard`: Vista protegida. Muestra nombre y datos devueltos por `/api/client/perfil`.
- `/admin/dashboard`: Vista protegida. Muestra métricas devueltas por `/api/admin/metricas`.

3. Implementación del `AuthContext`

- **Estados requeridos:** `user` (objeto con datos del usuario), `token` (string), `userType` (`'client'` | `'admin'` | `null`), y `isLoading`.
- **Persistencia:** Almacenar token en `localStorage`.
- **Métodos expuestos:**
  - `loginClient(email, password)`: Realiza petición Axios a `/api/auth/client/login`, guarda datos y redirige a `/cliente/dashboard`.
  - `loginAdmin(email, password)`: Realiza petición Axios a `/api/auth/admin/login`, guarda datos y redirige a `/admin/dashboard`.
  - `logout()`: Purga el estado global, limpia `localStorage` y redirige a `/`.

4. Componentes de Protección

- `<ClientRoute>`: Si no hay token, redirige a `/login`. Si hay sesión pero `userType !== 'client'`, redirige a `/admin/dashboard`.
- `<AdminRoute>`: Si no hay token, redirige a `/admin/login`. Si hay sesión pero `userType !== 'admin'`, redirige a `/cliente/dashboard`.

Ejercicio 3: Panel Admin con Roles Jerárquicos (RBAC)

Tipo: Fullstack (Back + Front)

Base: Ejercicios 1 y 2

1. Contexto y Objetivos

Ampliar la entidad `Admin` para dar soporte a un control de acceso basado en roles (*Role-Based Access Control*). El panel de administración ahora tiene tres tipos de operadores:

- `superadmin`: Acceso total.
- `gestor_productos`: Solo catálogo y stock.
- `auditor`: Solo lectura de reportes y métricas.

2. Modificaciones en Backend

- **Cambio en el modelo** `Admin`**:** Agregar columna `role` con valores tipo ENUM o String validado (`'superadmin'`, `'gestor_productos'`, `'auditor'`).
- **Middleware dinámico** `checkAdminRole(allowedRoles)`**:** Función que recibe un arreglo de roles permitidos y verifica si el rol contenido en el token coincide. Si no, devuelve HTTP `403`.
- **Endpoints a proteger:**
  - `GET /api/admin/usuarios` → Exclusivo `superadmin`.
  - `POST /api/admin/productos` → Habilitado para `superadmin` y `gestor_productos`.
  - `GET /api/admin/reportes` → Habilitado para `superadmin` y `auditor`.

3. Modificaciones en Frontend

- **Guardia Jerárquica** `<AdminRoleGuard allowedRoles="{['...']}">`**:** Envoltorio de rutas secundarias dentro del dashboard administrativo.
- **Navegación Dinámica (Sidebar / Menú lateral):**
  - Ocultar o deshabilitar enlaces si el administrador logueado no posee los permisos requeridos.
  - Si un `auditor` intenta tipear directamente la URL `/admin/usuarios`, el sistema debe redirigirlo a `/admin/sin-permiso` o mostrar un mensaje claro de acceso denegado.

Ejercicio 4: CRUD Completo con Paginación, Orden y Filtros

Tipo: Fullstack (Back + Front)

Base: Ejercicio 3

1. Contexto y Objetivos

Implementar la gestión del catálogo de productos garantizando un flujo eficiente de datos entre MySQL y React: el backend no debe volcar tablas masivas a la memoria, sino consultar únicamente la página y filtros solicitados.

2. Especificación Backend

- **Modelo** `Product` **(**`products`**):** `id`, `name`, `price` (float/decimal), `stock` (integer), `category` (string).
- **Endpoint** `GET /api/products` **(Público o de gestión):**
    - Query parameters obligatorios/opcionales:
        - `page` (default 1) y `limit` (default 5).
        - `search`: Filtrado por coincidencia parcial en `name` usando `Op.like`.
        - `category`: Filtrado por coincidencia exacta.
        - `sortBy` (`price` o `name`) y `order` (`ASC` o `DESC`).
    - Consulta mediante `Product.findAndCountAll(...)`.
- **Endpoints CRUD:** `POST`, `PUT /:id` y `DELETE /:id` protegidos por token de administración.
-

3. Especificación Frontend

- **Vista de Catálogo / Tabla:** Renderiza los productos devueltos por el backend.
- **Panel de Controles:**
    - Input de texto para búsqueda (dispara la consulta al presionar Enter o por botón).
    - `<select>` para filtrar por categorías.
    - Selector o botones para ordenar por precio (Mayor a menor / Menor a mayor).
- **Barra de Paginación:**
    - Botones "Anterior" y "Siguiente" deshabilitados en los extremos.
    - Indicador de página actual y total de páginas (`Página X de Y`).
    - Cada interacción actualiza los datos consumiendo la API sin recargar la ventana.

Ejercicio 5: Refresh Token, Persistencia F5 y Aislamiento de Sesiones

Tipo: Fullstack (Avanzado)

Base: Ejercicio 2 o 3

1. Contexto y Objetivos

Resolver dos de las fallas más comunes en aplicaciones web comerciales: el parpadeo/pérdida de sesión al presionar F5, y la colisión de credenciales cuando un usuario intenta operar como cliente y administrador en simultáneo en el mismo navegador.

2. Especificación Backend

- **Estrategia de tokens:**
    - **Access Token:** Corta duración (ej. 15 minutos).
    - **Refresh Token:** Larga duración (ej. 7 días). Se almacena en la tabla correspondiente (Client o Admin) o en una tabla auxiliar de sesiones.
- **Endpoints de control:**
    - POST /api/auth/refresh: Recibe el refresh token, valida existencia en la base de datos y firma, y emite un nuevo access token.
    - POST /api/auth/logout: Recibe el token y borra el refresh token de la base de datos para invalidar la sesión.

3. Especificación Frontend

- **Inicialización y Persistencia (Ciclo de vida F5):**
    - Al montar la aplicación (useEffect en el Provider), el Context debe leer el token almacenado y verificar su validez antes de marcar isLoading: false.
    - Evitar redirecciones espurias al login durante el milisegundo en que React procesa el estado inicial.
- **Aislamiento Estricto de Entornos:**
    - Si el usuario se encuentra autenticado como client e ingresa intencionalmente a /admin/login:
        - La vista debe detectar el conflicto y presentar una advertencia bloqueante: *"Ya posee una sesión de Cliente activa. Para ingresar al panel de Administración, debe cerrar su sesión actual"*.
        - Ofrecer botón directo para "Cerrar sesión de Cliente y continuar".
        - Queda terminantemente prohibido sobreescribir tokens en el almacenamiento local dejando estados cruzados en memoria.

Ejercicio 6: Contexto Avanzado — ThemeContext, Header Dinámico y Perfil

Tipo: 100% Frontend

Base: Frontend acumulado

1. Contexto y Objetivos

Demostrar dominio en la arquitectura de estados globales múltiples desacoplados, implementando un `ThemeContext` (modo claro/oscuro) y un componente `Header` completamente reactivo que transforma su diseño y opciones según el tipo de usuario conectado.

2. Implementación de `ThemeContext`

- **Estado:** `theme` (`'light'` o `'dark'`).
- **Persistencia:** Almacenado en `localStorage`. Si no existe valor previo, inicializar por defecto en `'light'`.
- **Efecto visual:** Añadir o remover dinámicamente la clase `.dark` en el elemento raíz `<html>` o `<body>` para habilitar el uso de estilos con Tailwind o CSS.
- **Función expuesta:** `toggleTheme()` que alterna entre ambos estados.

3. Componente `Navbar / Header` Reactivo

El menú principal debe suscribirse simultáneamente a `AuthContext` y `ThemeContext`, adaptando su estructura en tiempo real

Ejercicio 7: Layout Responsivo Mobile-First con Tailwind CSS

Tipo: 100% Frontend
Herramientas: React + Tailwind CSS

1. Contexto y Objetivos

Construir la pantalla principal (Home) de la tienda aplicando una metodología estricta **Mobile-First**. El sitio debe ofrecer una experiencia visual fluida y sin roturas de layout en dispositivos móviles (pantallas pequeñas), adaptándose gradualmente a tablets y escritorios.

2. Estructura y Secciones Requeridas

A. Barra de Navegación Responsiva

- **Móvil (`< 768px`):** Menú colapsado con botón tipo "hamburguesa". Al presionarlo, despliega un panel lateral (*drawer*) o menú desplegable vertical con los enlaces de navegación.
- **Escritorio (`>= 768px`):** El botón hamburguesa desaparece (`hidden md:flex`) y los enlaces se distribuyen horizontalmente de forma alineada.

B. Sección Hero (Banner Principal)

- Layout vertical centrado en móviles (imagen arriba, texto y botón de llamada a la acción abajo).
- En pantallas medianas y grandes (`md:` en adelante), se distribuye en dos columnas: texto a la izquierda, imagen representativa a la derecha.

C. Grilla de Categorías Destacadas

- Sistema de grillas adaptativo mediante Tailwind:
    - Móvil: 1 columna (`grid-cols-1`).
    - Tablet: 2 columnas (`sm:grid-cols-2`).
    - Escritorio: 4 columnas (`lg:grid-cols-4`).

- Cada tarjeta debe contar con efectos visuales de interacción (`hover:scale-105 transition-all`).

D. Vitrina de Productos

- Tarjetas de producto estructuradas con contenedor de imagen de proporción fija (`aspect-square` u `overflow-hidden`), badge de descuento, título, precio formateado y botón de compra.
- Adaptación de columnas: 1 columna en móvil chico, 2 en pantallas medianas, 3 o 4 en pantallas de escritorio.

E. Pie de Página (Footer)

- Distribución apilada en una columna en dispositivos móviles.
- Organización en tres o cuatro columnas alineadas (`Información`, `Enlaces Rápidos`, `Legal`) en pantallas de escritorio.